

3. Phase 1 UBL構文結合：環境定義、変換操作及び関数単位の処理

この文書は、**src/syntax_binding.py** を対象とし、Phase 1の入力定義、構文結合表、順方向変換、逆変換、xBRL-CSVメタデータ、検証及び関数単位の処理をまとめます。

処理の流れ

```
UBL Invoice XML
→ 結合表・LHMレイアウト読み込み
→ XPath / selector評価
→ ディメンション所有関係の解決
→ 疎な構造化CSV行の生成
→ xBRL-CSVメタデータ生成
→ (逆変換) UBL XML再構築及び要素順整列
```

Phase 1の完全な仕様及びユーザーガイド

1. 目的

この文書は、UADC概念実証（PoC）で使用する構文結合変換プログラムを規定します。

このプログラムは、構文結合CSVを適用し、XMLインボイス文書をディメンションに基づく階層型CSVへ変換します。順方向変換では、CSV列と生成済みタクソノミとの関係を定義するJSONメタデータも出力します。同じ結合情報を使用して、階層型CSVからXMLへ逆変換することもできます。

実装レベルの処理の詳細は、この文書の第7章から第10章まで及び第15章に記載しています。

この文書に示すパスは、リポジトリをプッシュ又はクローンした後の **UADC_PoC** 作業ディレクトリからの相対パスです。ただし、**../** で始まるパスは、**UADC_PoC** と同じ階層にあるディレクトリを示します。

2. メインプログラム

プログラム：

```
src/syntax_binding.py
```

この変換プログラムは、運用用の階層変換で使用する、対応済みの名前空間処理及びXPath支援関数を内部に備えています。

3. 入力ファイル

3.1 XML入力

XML入力には、UBL Invoice XML文書を使用します。

PoCサンプル :

```
samples/input/openpeppol_ubl_invoice_minimal.xml
```

パッケージサンプル :

```
../syntax_binding_revised_package/invoice.xml
```

3.2 構文結合CSV

結合CSVは、セマンティックモデルのパスをXMLのXPath式へ対応付けます。

PoC用結合ファイル :

```
specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv
```

パッケージ用結合ファイル :

```
../syntax_binding_revised_package/bindings.csv
```

運用上必要な結合列 :

```
sequence
level
structured_csv_level
type
name
datatype
multiplicity
semantic_path
structured_csv_column
element
xpath
```

semantic_path と **xpath** の両方が設定されていない行は、ファクト抽出の対象外とします。
type=C の行は、セマンティッククラスのコンテキスト及び繰返し行スコープを定義します。

type=A の行は、抽出又は生成する値を定義します。

3.3 結合表に組み込まれたLHMレイアウト

構文結合CSVは、必要なLHM列を複写し、LHMの行順を維持して作成します。そのため、変換プログラムは結合表自体から、次の情報を取得します。

- 出力列の順序
- BGのディメンション列
- BTの値列
- セマンティックパスから最終的なLHM **element** 名への対応
- **multiplicity** に基づく繰返しBG
- 利用可能な場合は、BG行の繰返しコンテキストXPath

これらの列を管理する元データは、次のLHMです。

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

3.4 テンプレートCSV

--template-csv を指定した場合、変換プログラムは、そのヘッダーを出力列順として使用し、値が設定されたテンプレート行から、フィールドとディメンションの配置の一部を推定します。

パッケージ用テンプレート：

```
../syntax_binding_revised_package/invoice.csv
```

--template-csv を指定した場合でも、主なフィールド順は結合表から導出したレイアウトに従い、テンプレートはフィールド配置の補助情報として使用されることがあります。

4. 順方向変換の出力

出力は、UTF-8-SIGで符号化したCSVファイルです。

PoC出力：

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
```

パッケージ出力：

```
out/phase1/<input-xml-stem>.csv
```

出力には、次の内容が含まれます。

- ルートディメンション列としての **dlInvoice**
- 繰返しBGごとの追加の **d*** 列
- ディメンション列に続くBT値列
- インボイスレベルのファクトを格納する一つのルート行
- 結合定義によって値が生成される繰返しBGコンテキストごとの一つの行

順方向変換では、JSONメタデータも出力します。既定のメタデータファイル名は、次のとおりです。

```
<output-csv-stem>.json
```

例：

```
out/phase1/openpeppol_ubl_invoice_minimal.json
```

このメタデータは、Arelleが **loadFromOIM** プラグインで読み込めるxBRL-CSVメタデータ文書です。構造化CSVのテーブルを、生成済みxBRL-CSVタクソノミのエントリーポイントへ結び付け、CSV列をタクソノミの概念へ対応付けます。レポートの **documentInfo.taxonomy** プロパティを空にしてはなりません。**en16931-oim** が存在しない場合は、変換エラーとして扱います。JSONメタデータは、xBRL-CSV OIMタクソノミのエントリーポイントとして **out/taxonomy/pli/en16931-oim-.xsd** を参照します。

5. 逆変換の出力

--reverse を指定した場合、入力階層型CSV、出力はXMLファイルになります。

逆変換の出力例：

```
out/reverse/en16931_reverse_invoice.xml
```

逆変換プログラムは、次の処理を行います。

- 階層型CSVの行を読み込む。
- 構文結合CSVに基づいてフィールドを解決する。
- 結合表から導出したレイアウトを使用して、フィールドをディメンション行へ対応付ける。
- 結合済みXPath式からUBL Invoice XML要素を作成する。
- 結合済みの値を持つ繰返しディメンション行ごとに、一つのXMLコンテキストを作成する。
- インボイスの通貨フィールドから、金額要素に必要な **currencyID** 属性を導出する。
- セマンティック用語が繰返しクラスに属していても、その構文上の絶対XPathが現在のコンテキスト外を指す場合は、文書ルートを基点として処理する。

- BT-90を **AccountingSupplierParty** の下に出だし、**PaymentMeans** の下に入れ子の **Invoice** を作成しない。

6. コマンドラインインターフェース

```
syntax_binding.py XML_FILE -b BINDING_CSV -o OUTPUT_CSV [options]
```

逆変換形式：

```
syntax_binding.py INPUT_CSV --reverse -b BINDING_CSV -o OUTPUT_XML  
[options]
```

引数：

- **XML_FILE**：順方向モードで使用する入力XMLファイル。
- **INPUT_CSV**：逆変換モードで使用する入力階層型CSVファイル。
- **-b、--binding**：構文結合CSV。
- **-o、--output**：順方向モードでは出力階層型CSV、逆変換モードでは出力XML。
- **--template-csv**：列順及びディメンション配置を定義する任意のCSVテンプレート。
- **--metadata-output**：JSONメタデータの出力パス。既定値は、出力CSVの拡張子を **.json** に変更したパスです。
- **--taxonomy-base**：JSONメタデータから参照する生成済みタクソノミの基点ディレクトリ。既定値は **out/taxonomy** です。
- **--reverse**：階層型CSVをXMLへ逆変換します。
- **--drop-empty-columns**：生成したすべての行で値が設定されていない列を削除します。
- **--d-invoice**：**dInvoice** 列へ書き込む値。既定値は **1** です。
- **-e、--encoding**：CSVの文字エンコーディング。既定値は **utf-8-sig** です。

7. 順方向変換の処理モデル

7.1 名前空間の処理

変換プログラムは、**xml.etree.ElementTree.iterparse** を使用して、入力XMLに宣言された名前空間を収集します。

XPath支援関数は、次のような名前空間接頭辞付き要素に対応します。

```
/Invoice/cbc:ID  
/Invoice/cac:AccountingSupplierParty/cac:Party/cac:PartyName/cbc:Name
```

7.2 結合表に組み込まれたLHMレイアウトの読み込み

構文結合表を読み込むときは、次の規則を適用します。

1. C行をBG又はクラス行として扱う。
2. C行がインボイスのルートであるか、**0..*** 又は **1..*** のような繰返し多重度を持つ場合に限り、ディメンションとする。
3. 繰返さないBG行は、セマンティック上のグループ化ノードとして保持し、ディメンションとしては出力しない。
4. ディメンション名は、**d** + UpperCamelCase(**element**) で生成する。
5. A行をBT又はファクト行として扱う。
6. ファクト列名は、**structured_csv_column** から取得する。
7. 各ファクトを、最も近い繰返し祖先ディメンションへ割り当てる。

多重度の上限が次の値の場合、繰返しとして扱います。

```
*
n
unbounded
```

又は、上限が2以上の数値である場合も繰返しとして扱います。

7.3 結合定義の解析

各結合行は、次の項目を持つ内部結合レコードへ変換されます。

```
semantic_path
xpath
root
dimension
filter_field
filter_value
field
```

変換プログラムは、次の形式に対応します。

- **\$.invoice.invoiceNumber** のようなルートレベルのパス
- 任意のフィルタ述語を持つディメンションパス
- 最も近いLHMディメンションを使用して解決する、一般的な入れ子のセマンティックパス

7.4 行の生成

順方向変換では、繰返しグループを個別に処理するのではなく、XMLの親コンテキストを再帰的にたどります。変換プログラムは、次の行を作成します。

- インボイスレベルのファクトを格納するルートインボイス行
- VAT内訳又はインボイス明細など、繰返しコンテキストごとのBG行

繰返しBG行については、次の処理を行います。

1. **type=C** 行から **BindingClass** ツリーを構築する。
2. ルート呼出しを **dlInvoice=1** で開始する。
3. 現在のクラスが直接所有する **type=A** ファクトを、現在のXMLコンテキストからの相対XPathで抽出する。
4. 繰り返さない子クラスは、現在の構造化CSV行を再利用する。
5. 繰返し子クラスは、現在の親XMLコンテキストの下で、発生ごとに処理する。
6. 繰返しの各発生には、**dlInvoiceLine=2** のように、1を起点とするディメンション値を割り当てる。
7. 入れ子になった繰返し行へ、祖先の繰返しディメンション値を複写する。

繰り返さない子クラスは、親行へ平坦化します。繰返し子クラスは親行へ平坦化せず、最初の発生を含むすべての発生を、独立した子行へ書き出します。

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

したがって、親行の **b1**、**b2** 及び **b3** は空欄になります。**1,1,a1V1,a2V1,b1V1,b2V1,b3V1** のような行は、**dAaa** が所有するファクトと、繰返し **dBbb** スコープが所有するファクトを混在させるため、無効です。

7.5 列の処理

LHMを使用する場合、列順は次のとおりです。

```
ディメンション列、続いてファクト列
```

LHMを使用せずテンプレートだけを使用する場合は、テンプレートのヘッダーが出力順を制御します。

LHMにもテンプレートにもヘッダーがない場合は、結合定義からフィールド名を導出します。

8. メタデータJSONの処理モデル

順方向モードでは、構造化CSVを生成した後にJSONメタデータを書き出します。

メタデータの構造は、xBRL-CSVレポートパッケージの形式に従います。

- **documentInfo** : xBRL-CSVの文書型、名前空間及びタクソノミ・エントリーポイントへの参照。
- **tables** : 生成した構造化CSVテーブル及びその相対URL。
- **tableTemplates** : テーブルレベルのディメンション及び列ごとの概念対応。

ディメンション列は、テーブルディメンションとして宣言します。

```
"plt:d_en16931_Invoice": "$dInvoice"
```

ファクト列は、xBRLの概念を指定して宣言します。

例：

```
{
  "InvoiceNumber": {
    "dimensions": {
      "concept": "en16931:InvoiceNumber"
    }
  }
}
```

LHMのデータ型が金額を示す場合、メタデータでは金額ファクトにテスト用の既定単位 **iso4217:JPY** を設定します。実運用では、該当する通貨項目から単位を導出してください。

9. 逆変換の処理モデル

逆変換モードでは、次の処理を行います。

1. **csv.DictReader** を使用して、入力階層型CSVを読み込む。
2. 順方向モードと同じ方法で、結合表から導出したレイアウト及び結合CSVを読み込む。
3. XMLを作成する前に、各ファクトを、その行で値が設定された最も深いディメンションと照合する。祖先ディメンションが所有するファクトが繰返し子行にある場合は拒否し、親と子を混在させた行を逆変換しない。
4. ルートレベルのフィールドを、インボイスルート直下のXMLパスへ書き込む。
5. ディメンションフィールドを、その **d*** ディメンション列によってグループ化する。
6. 結合済みの値を持つ各ディメンション行について、BGのXPathから繰返しXMLコンテキストを作成する。
7. 結合済みフィールドの値を、そのコンテキストからの相対XPathへ書き込む。
8. **cbc:ChargeIndicator=false()** 又は **cbc:DocumentTypeCode='130'** のようなXPath述語は、対応するXMLコンテキストを作成するときに、子要素の値として実体化する。
9. 金額要素の **currencyID** には、インボイスの文書通貨を設定する。税務会計通貨用TaxTotalパスの **currencyID** には、税務会計通貨を設定する。
10. **relative_xpath(binding.xpath, repeat_path)** の結果が絶対パスのままである場合、その結合は繰返し構文コンテキストの外側にあるため、文書ルートを基点として書き込む。繰返し要素に内包されるパスだけを、その要素からの相対パスとして書き込む。
11. スキーマ検証に必要でありながらEN 16931のBTではないUBL補助要素を、必要に応じて追加する。これには **cac:TaxScheme/cbc:ID** 及び不足している **cbc:ChargeIndicator=false** が含まれる。

12. 生成したUBL子要素を、UBLスキーマの順序へ正規化する。--ubl-schema-root 又は --ubl-schema-url を指定した場合は、XSDの **xs:sequence** 宣言から子要素の順序を導出する。スキーマの参照元を指定しない場合は、対応済みInvoice構造の組込み順序を代替として使用する。

現在の逆変換プログラムは、PoCのUBL Invoice結合パターンを対象としています。正準XMLを再現するためではなく、構造化CSV表現のラウンドトリップを検証するための機能です。

10. 通貨属性の規則

構文結合では、**currencyID** を、セマンティックな通貨項目から導出する構文メタデータとして扱います。

- BT-5 **DocumentCurrencyCode** : 通常のインボイス金額要素に設定する既定の **currencyID**。
- BT-6 **TaxAccountingCurrencyCode** : 税務会計通貨用TaxTotal分岐に設定する **currencyID**。
- BT-110及びBG-23 : UBLパス述語 **cbc:TaxAmount/@currencyID=/Invoice/cbc:DocumentCurrencyCode** を使用する。
- BT-111 : UBLパス述語 **cbc:TaxAmount/@currencyID=/Invoice/cbc:TaxCurrencyCode** を使用する。

順方向変換では、これらの絶対参照パスを文書ルートから評価します。**Allowance-example.xml** では、BT-110は **1225.00 EUR**、BT-111は **9324.00 SEK** として解決されます。

BT-90は、スコープをまたぐ逆方向の対応例です。そのセマンティックパスは支払指示／口座振替の下にあります。UBL XPathは **/Invoice/cac:AccountingSupplierParty/.../cbc:ID** を指します。そのため逆変換では、**PaymentMeans/Invoice/AccountingSupplierParty** を構築せず、XPathを文書ルート基点のまま処理します。

CSVには、セマンティックな金額値と通貨コード値を別々に格納します。逆変換時に、必要なUBL **currencyID** 属性を書き込みます。

11. 制約

- 変換プログラム自体では、XMLをUBLスキーマに対して検証しません。回帰テスト **tests/test_roundtrip_xml_ubl_schema.py** が、UBL 2.1スキーマ検証を行います。
- EN 16931のビジネスルールを検証しません。
- 変換プログラム内部ではArelleを実行しません。回帰テスト **tests/test_xbrl_csv_metadata_arelle.py** が、生成したメタデータをArelleで検証します。
- XPath 2.0の全機能を評価しません。
- 対応する述語は、PoCの結合定義で使用する事例に意図的に限定しています。
- テンプレートの列だけでは行を作成しません。行を生成するには、対応する結合定義が必要です。
- 逆変換XMLの要素順は、指定されたUBL XSDスキーマのルート又はUBL InvoiceスキーマURLから導出できます。**syntax_binding.py** の組込み子要素順序表は、スキーマの参照元を指定しない場合だけ使用する代替手段です。
- 逆変換XMLでは、結合対象外のXML内容が省略される場合があります。

- 逆変換XMLは、結合済みCSV値から生成するものであり、元のXMLとバイト単位で同一にすることを目的としていません。

12. 回帰テスト

PoC LHM駆動変換テスト：

```
tests/test_lhm_hierarchical_csv_layout.py
```

パッケージのテンプレート／結合変換テスト：

```
tests/test_syntax_binding.py
```

OpenPeppol構造化変換テスト：

```
tests/test_openpeppol_invoice_conversion.py
```

逆変換ラウンドトリップテスト：

```
tests/test_syntax_binding_reverse.py
```

ラウンドトリップ成果物及びメタデータのテスト：

```
tests/test_roundtrip_artifacts.py
```

13. 対象外の機能

構文結合変換プログラムは、次の処理を行いません。

- LHM定義の生成
- XBRLタクソノミファイルの生成
- フラットCSVへのセマンティック結合の実行
- EN 16931又はOpenPeppolの完全な検証
- リポジトリの同期又は公開処理

14. 運用手順

14.1 Phase 1入力の準備

元となるUBLインボイスを設定済みの入力ディレクトリへ配置し、対応する構文結合CSVを選択します。**type=C**の結合行がクラツリーを定義し、**type=A**の行がファクトの抽出及び逆方向の書込み規則を定義します。結合表は実行時の基準となるため、処理前にそのバージョンを確認してください。

14.2 変換及び確認

一つのXML文書又は入力ディレクトリに対して、**src/syntax_binding.py**を実行します。生成した構造化CSVについて、疎な親行と繰返し子行を確認し、JSONメタデータについて、タクソノミ・エントリーポイント、ディメンション及び単位を確認します。必要な行パターンは、**02_STRUCTURED_CSV_LHM_BINDINGS.md** 第4章に規定しています。

14.3 逆変換及び検証

逆変換モードを使用してUBLを再構築します。変換プログラムは文書ルートから開始し、スキーマ順に絶対XPathの祖先を作成し、述語及び通貨属性を適用し、値が空の小数要素を出力しません。生成したXMLをUBLスキーマで検証し、そのXMLをもう一度順方向変換して、元の構造化CSVと比較します。

15. 関数単位の処理仕様

15.1 結合定義及びレイアウトの読み込み

- **read_binding_rows** : 行順を失わずに結合CSVを読み込みます。
- **build_layout_from_rows** : 列、ディメンションの所有関係、クラスの多重度及び結合表に組み込まれたLHMレイアウトを導出します。
- **read_bindings** : 属性行を結合オブジェクトへ変換し、セマンティックパスを解決します。
- **build_binding_class_tree** : 順方向及び逆方向の両方で使用する、入れ子のクラスモデルを構築します。
- **direct_class_fields** 及び **walk_binding_classes** : ファクトを所有クラスへ関連付け、一定の順序でクラスをたどります。

15.2 XPathの評価

- **collect_namespaces** : 入力XMLに宣言された名前空間を記録します。
- **xml_split_xpath**、**xml_split_step_predicate** 及び **xml_split_terminal_attribute** : 結合XPathを安全に分解します。
- **find_nodes**、**xml_predicate_matches** 及び **get_value** : 述語内の文書ルート参照を許容しながら、現在のクラスコンテキストを基準としてXPathを評価します。
- **infer_repeat_path**、**common_xpath_prefix** 及び **relative_xpath** : 繰返しクラスのXMLノードスコープを導出します。

15.3 順方向の行出力

- **write_hierarchical_csv** : XMLの再帰的な走査、ディメンション番号の割当て、疎な行の出力、列順及びメタデータの作成を統括します。

- **new_row** : 祖先ディメンションを設定した新しい行を作成します。
- **row_has_values** : 構造だけで値を持たない行の出力を防ぎます。
- **validate_hierarchical_row_scopes** : 繰返し子の最初の値が親行へ統合されている場合にエラーとします。
- **drop_empty_columns** : 必要なディメンションを保持したまま、使用されていない値列を削除します。

単一の子クラスについては、抽出したフィールドを最も近い繰返し祖先の行に保持します。繰返し子クラスについては、親行を別に出し、発生番号1を含むすべての子を、それぞれ独立したディメンション行として出力します。

15.4 メタデータの生成

- **binding_column_metadata** : 結合定義から、概念、データ型、ディメンション及び単位の情報を出します。
- **taxonomy_entrpoints** : 日付付きのタクソノミ・エントリーポイントを選択します。
- **xbml_csv_column_definition** : OIMに適合する列定義を作成します。
- **write_csv_metadata** : JSONメタデータ及び相対テーブルパスを書き出します。

15.5 逆方向のXML構築

- **write_xml_from_hierarchical_csv** : ディメンションスコープによって行をグループ化し、XMLの再構築を制御します。
- **split_xml_path**、**ensure_path** 及び **find_or_create_child** : UBLルートからの絶対XPathを処理します。子孫全体から一致する要素を検索することはありません。
- **set_xml_value**、**set_relative_xml_value** 及び **set_xml_value_with_currency** : 元の値が空でない場合だけ値を書き込みます。
- **apply_currency_attribute** 及び **resolve_currency_references** : 条件付き金額について、文書通貨と税務会計通貨を区別します。
- **load_ubl_child_order** 及び **sort_children_for_ubl_schema** : 直列化の前に、要素をUBLスキーマ順へ並べ替えます。

15.6 検証の境界

変換プログラムは、階層型行の仕様を検証し、スキーマ構造に沿ったXMLを再構築します。ただし、外部のUBLスキーマ検証は必須のテスト工程です。EN 16931及びPeppol Schematronなどのビジネスルール検証は変換プログラムの対象外であり、周辺の処理フローで実行してください。

16. 構文結合変換の概要

構文結合変換文書

この文書では、XMLから構造化CSVへの変換及び構造化CSVからXMLへの変換プログラムを説明します。

XPathの親コンテキスト処理、セマンティックパスの解決、**dInvoice** 及び **dInvoiceLine** ディメンションの処理、内部の **dict/list/dataclass** オブジェクト並びに関数単位のデータフローに関する詳

細な実装説明については、**02_STRUCTURED_CSV_LHM_BINDINGS.md** 第15章を参照してください。この文書は、構文結合コマンドのプログラム仕様書及び運用ガイドです。

ファイル

- **program_specification.md** : 変換プログラムの入力、出力、ディメンションの動作、JSON メタデータの生成、逆変換、通貨処理、XPathセクタ処理及び対象外の機能を定義します。
- **user_guide.md** : 順方向変換、逆変換、ラウンドトリップ成果物及びトラブルシューティングのコマンド例を示します。

関連ディレクトリ

- **../src/** : 変換プログラムの実装。特に **syntax_binding.py**。
- **../specs/bindings/syntax/** : 現在使用しているUBL Invoice構文結合CSV。
- **../specs/lhm/** : 構造化CSV列及びタクソノミ概念の定義に使用するLHM CSV。
- **../tests/roundtrip/** : レビュー用の元XML、構造化CSV、メタデータJSON及び再生成XML。

Phase 1では、EN 16931の構文結合を安定した基準として使用します。OpenPeppol CIUSの検証は、後続のオーバーレイとして追加する予定です。

17. 構文結合ユーザーガイド

ユーザーガイド : 構文結合によるXMLから階層型CSVへの変換

1. 作業ディレクトリ

コマンドは、**UADC_PoC** ディレクトリで実行します。

```
cd UADC_PoC
```

以下のパスは、**../** で始まるものを除き、このディレクトリからの相対パスです。

ローカルのWindows環境では、Pythonコマンドを次のように設定します。

```
$python = 'python'
```

2. メインスクリプト

次のスクリプトを使用します。

```
src/syntax_binding.py
```

このスクリプトは、構文結合CSVを使用して、XMLをディメンションに基づく階層型CSVへ変換します。また、**--reverse** を指定することで、階層型CSVからXMLへの逆変換にも対応します。

XPathの親コンテキストの再帰処理、セマンティックパスの解決、**dInvoice** 及び **dInvoiceLine** の割当て並びに関数単位の日付フローなど、実装の詳細については、

02_STRUCTURED_CSV_LHM_BINDINGS.md 第15章を参照してください。

3. PoC OpenPeppolサンプルの変換

次のコマンドを実行します。

```
& $python .\src\syntax_binding.py `
.\samples\input\openpeppol_ubl_invoice_minimal.xml `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
--metadata-output .\out\phase1\openpeppol_ubl_invoice_minimal.json `
--taxonomy-base .\out\taxonomy `
-o .\out\phase1\openpeppol_ubl_invoice_minimal.csv
```

出力：

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
out/phase1/openpeppol_ubl_invoice_minimal.json
```

このコマンドは、構文結合CSVに組み込まれたLHM由来の列を使用して、出力ディメンション及びファクト列を決定します。また、CSV列と生成済みタクソノミとの関係を定義するJSONメタデータも作成します。

変換プログラムは、**lhm_level** を構造化CSVの実効的な階層として扱います。繰り返さないSeller又はBuyerグループなど、**lhm_level** が空欄のBG行は、セマンティック上のグループ化ノードとしてだけ使用し、そのBT値を最も近い祖先ディメンション行へ書き込みます。

4. 改訂パッケージサンプルの変換

次のコマンドを実行します。

```
& $python .\src\syntax_binding.py `
..\syntax_binding_revised_package\invoice.xml `
-b ..\syntax_binding_revised_package\bindings.csv `
--template-csv ..\syntax_binding_revised_package\invoice.csv `
```

```
--metadata-output .\out\phase1\package_invoice_hierarchical.json `
-o .\out\phase1\package_invoice_hierarchical.csv
```

出力：

```
out/phase1/package_invoice_hierarchical.csv
out/phase1/package_invoice_hierarchical.json
```

このコマンドは、パッケージのテンプレートCSVを使用して、想定する出力列順を維持します。

5. コマンドオプション

基本形式：

```
& $python .\src\syntax_binding.py XML_FILE `
-b BINDING_CSV `
-o OUTPUT_CSV
```

オプション：

- **--template-csv**：テンプレートCSVのヘッダーを使用して出力列順を定義します。
- **--metadata-output**：JSONメタデータの出力パスを設定します。既定値は、出力CSVの拡張子を **.json** に変更したパスです。
- **--taxonomy-base**：JSONメタデータから参照する生成済みタクソノミのディレクトリを設定します。既定値は **out/taxonomy** です。
- **--reverse**：階層型CSVをXMLへ逆変換します。
- **--ubl-schema-root**：展開済みUBL XSDファイルを含むローカルディレクトリ。逆変換モードでは、XSDの **xs:sequence** 宣言から子要素順を導出します。
- **--ubl-schema-url**：UBL Invoice XSDのURL。逆変換モードでは、このURLからimport又はincludeされたスキーマをたどり、XSDの **xs:sequence** 宣言から子要素順を導出します。
- **--drop-empty-columns**：値が一つもない出力列を削除します。
- **--d-invoice**：**dInvoice** の値を設定します。既定値は **1** です。
- **-e、--encoding**：CSVの文字エンコーディング。既定値は **utf-8-sig** です。

6. 階層型CSVからXMLへの逆変換

最初に階層型CSVを作成するか、既に存在することを確認します。

```
& $python .\src\syntax_binding.py `
.\samples\input\openpeppol_ubl_invoice_minimal.xml `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
```

```
--metadata-output .\out\phase1\openpeppol_ubl_invoice_minimal.json `
-o .\out\phase1\openpeppol_ubl_invoice_minimal.csv
```

次にXMLへ逆変換します。

```
& $python .\src\syntax_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
--reverse `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
-o .\out\reverse\en16931_reverse_invoice.xml
```

出力：

```
out/reverse/en16931_reverse_invoice.xml
```

ローカルのUBLスキーマパッケージから子要素順を導出する場合は、**--ubl-schema-root** を追加します。

```
& $python .\src\syntax_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
--reverse `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
--ubl-schema-root .\out\cache\UBL-2.1\xsd `
-o .\out\reverse\en16931_reverse_invoice.xml
```

オンラインのUBL Invoiceスキーマ・エントリーポイントから子要素順を導出する場合は、**--ubl-schema-url** を使用します。変換プログラムは、そのURLからimport又はincludeされたスキーマをたどります。

逆変換XMLは、結合済みCSV値から生成します。ラウンドトリップ検証を目的としており、結合対象外のXML内容は再現できない場合があります。LHMに **syntax_sequence** がある場合、逆変換出力はその値に従ってUBLスキーマ順を保持します。

逆変換時に、金額の **currencyID** 属性を導出します。

- 通常のインボイス金額要素には **DocumentCurrencyCode** を使用する。
- 税務会計通貨用TaxTotal分岐には **TaxAccountingCurrencyCode** を使用する。

順方向変換でも、これらの通貨項目を使用してTaxTotalの分岐を区別します。**Allowance-example.xml** では、BT-110は **currencyID** が **DocumentCurrencyCode** と一致するTaxAmount (**1225.00 EUR**) を選択し、BT-111は **currencyID** が **TaxAccountingCurrencyCode** と一致するTaxAmount (**9324.00 SEK**) を選択します。

一部のセマンティックな子項目は、繰返しUBL構文コンテキストの外側に格納されます。BT-90は、意味上は支払指示／口座振替に属しますが、そのUBL XPathは **AccountingSupplierParty** の下を指します。逆変換では、現在の繰返しXPathに内包されない絶対結合XPathを、文書ルートから書き込みます。**PaymentMeans** の下に入れ子の **Invoice** を生成してはなりません。

7. メタデータを含むテスト成果物の生成

ラウンドトリップテストの成果物は、次のディレクトリに生成します。

```
tests/roundtrip/
```

次のコマンドを実行します。

```
& $python .\tools\build_roundtrip_test_artifacts.py
```

生成するディレクトリ構成は、次のとおりです。

```
tests/roundtrip/<dataset>/
  source_xml/
  structured_csv/
  metadata_json/
  roundtrip_xml/
```

各ディレクトリの内容：

- **source_xml**：元のUBL Invoice XML。
- **structured_csv**：階層型構造化CSV。
- **metadata_json**：CSV列とタクソノミとの関係を定義するJSONメタデータ。
- **roundtrip_xml**：構造化CSVから再生成したXML。

スクリプトは、**--metadata-output** を使用して、メタデータJSONを **metadata_json/** に配置します。

8. 入力結合CSV

結合CSVには、次の列を含めます。

```
semantic_path,xpath
```

例：

```
$.invoice.invoiceNumber,/Invoice/cbc:ID
$.invoice.invoiceIssueDate,/Invoice/cbc:IssueDate
```

次の互換列名も使用できます。

```
path
source_xpath
xml_path
```

9. 出力CSVレイアウト

階層型CSVでは、次の列を使用します。

- インボイスのルートを示す **dInvoice**
- 繰返しBGを示す **d*** デイメンション列
- BT値を格納するファクト列

LHMを使用する場合は、デイメンション列を先頭に配置し、その後にBT列を配置します。

出力例：

```
dInvoice,dVatBreakdown,dInvoiceLine,InvoiceNumber,InvoiceIssueDate,...
1,,,INV-2026-0001,2026-07-06,...
1,1,,,,...
1,,1,,,,...
```

親 **dAaa** と子 **dBbb** があり、子が繰り返されない場合は、子を親行へ平坦化します。

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,b1V1,b2V1,b3V1
```

dBbb が繰り返される場合、親行ではすべての子ファクトを空欄とし、最初の子の発生から独立した行に格納します。

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

最初の子行に、親ファクトと繰返し子ファクトと一緒に格納しないでください。逆変換では、このような混在行レイアウトをエラーとして報告します。

10. JSONメタデータの構成

メタデータファイルは、xBRL-CSVメタデータ文書です。主なセクションは、次のとおりです。

- **documentInfo** : xBRL-CSVの文書型及びタクソノミ参照。
- **tables** : 生成した構造化CSVテーブルのURL。
- **tableTemplates** : テーブルレベルのディメンション及びファクト概念の対応。

主要な列が、次のように対応付けられていることを確認します。

```
tableTemplates.structured.dimensions["plt:d_en16931_Invoice"] ->
"$dInvoice"
tableTemplates.structured.columns.InvoiceNumber.dimensions.concept ->
"en16931:InvoiceNumber"
tableTemplates.structured.columns.DocumentCurrencyCode.dimensions.concept
-> "en16931:DocumentCurrencyCode"
```

11. 回帰テストの実行

LHM駆動のOpenPeppolサンプル変換を確認します。

```
& $python .\tests\test_lhm_hierarchical_csv_layout.py
```

改訂パッケージの変換を確認します。

```
& $python .\tests\test_syntax_binding.py
```

OpenPeppol構造化変換を確認します。

```
& $python .\tests\test_openpeppol_invoice_conversion.py
```

逆変換のラウンドトリップを確認します。

```
& $python .\tests\test_syntax_binding_reverse.py
```

ラウンドトリップ成果物及びメタデータを確認します。

```
& $python .\tests\test_roundtrip_artifacts.py
```

生成したxBRl-CSVメタデータをArelleで検証します。

```
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```

再生成したラウンドトリップXMLをUBL 2.1 Invoiceスキーマに対して検証します。

```
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

12. トラブルシューティング

使用可能な結合定義が見つからない

結合CSVに **semantic_path** と **xpath** の値が設定されていることを確認してください。

出力列が不足している

--drop-empty-columns を使用すると、生成した値が一つもない列は削除されます。常に同じ完全なヘッダーが必要な場合は、このオプションを指定せずに実行してください。

メタデータJSONからタクソノミを参照できない

変換プログラムは、空でない **documentInfo.taxonomy** 配列を書き出す必要があります。最初にタクソノミを生成してください。

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

又は、**plt/en16931-oim-*.xsd** を含むディレクトリを **--taxonomy-base** で指定してください。このスキーマが存在しない場合、空のタクソノミー一覧を書き出すのではなく、メタデータ生成をエラーとします。

繰返し行が生成されない

次の点を確認してください。

- LHMのBG行が、**0..*** 又は **1..*** のような繰返し多重度を持っていること。
- BG行のXPathが、繰返しXMLコンテキストを指していること。
- 構文結合CSVに、そのBGセマンティックパスの下に結合定義が含まれていること。

値がディメンション行ではなくインボイスのルート行に書き込まれる

構文結合CSVに、LHM由来の **type**、**multiplicity**、**semantic_path** 及び **structured_csv_column** 列が含まれていることを確認してください。変換プログラムは、結合表自体から、各BTセマンティックパスに最も近い繰返しC行ディメンションを解決します。

パッケージサンプルの明細行が空になる

テンプレート列だけでは行は生成されません。インボイス明細行を生成するには、インボイス明細の結合定義が必要です。

逆変換XMLの要素順が元ファイルと一致しない

逆変換XMLは、結合行及び階層型CSVの値から再構築します。XML要素順を検証する必要がある場合は、UBLスキーマからLHMの **syntax_sequence** を設定してください。結合定義又は固定値規則で表現されていないXML内容は、引き続き再現できません。

内部処理の詳細解説

Phase 1順方向変換：UBL XMLから構造化CSVへ

スクリプト：

```
src/syntax_binding.py
```

主な関数：

```
write_hierarchical_csv(...)
```

入力：

- UBL Invoice XML
- 構文結合CSV。現在は **specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv**

出力：

- Phase 1構造化CSV
- xBRL-CSVメタデータJSON

結合表の解釈

構文結合表には、クラス行とファクト行の両方があります。

クラス行：

```

type=C
semantic_path=$.invoice.invoiceLine
structured_csv_column=InvoiceLine
multiplicity=1..*
xpath=/Invoice/cac:InvoiceLine

```

ファクト行：

```

type=A
semantic_path=$.invoice.invoiceLine.invoiceLineIdentifier
structured_csv_column=InvoiceLineIdentifier
xpath=/Invoice/cac:InvoiceLine/cbc:ID

```

クラス行は、XMLコンテキストの位置と、そのクラスが繰り返すかどうかを示します。ファクト行は、そのコンテキスト内から抽出する値を示します。

内部データ構造

read_binding_rows(binding_csv, encoding) は、結合表を次の形式でそのまま読み込みます。

```
list[dict[str, str]]
```

各辞書はCSVの一行で、見出し名をキーとします。

read_binding_layout(...) と **build_layout_from_rows(...)** は、結合表から実行時の構造化CSVレイアウトを表す **BindingLayout** を構築します。

BindingLayout の主な項目：

- **fieldnames:** **list[str]** : 最終的な構造化CSV列順
- **dimensions:** **list[str]** : ディメンション列
- **field_dimension:** **dict[str, str]** : 値列と、その値を所有するディメンション列の対応
- **field_by_semantic_path:** **dict[str, str]** : セマンティックパスと構造化CSV値列の対応
- **semantic_path_dimension:** **dict[str, str]** : セマンティッククラスパスとディメンション列の対応
- **dimension_xpath:** **dict[str, str]** : ディメンション列とXMLコンテキストXPathの対応
- **dimension_ancestors:** **dict[str, list[str]]** : 繰り返し祖先ディメンション
- **dimension_repeats:** **dict[str, bool]** : 出力する各ディメンションの繰り返し状態
- **syntax_sequence_by_field**、**syntax_sequence_by_dimension** : 逆方向XML出力時の並べ替え情報

read_bindings(...) は、次を構築します。

```
bindings: list[Binding]
```

Binding は、**type=A** の行を正規化したdataclassです。次を保持します。

- **order**
- **semantic_path**
- **xpath**
- **field** : 構造化CSVの値列
- **dimension** : 行スコープを表すディメンション列
- 任意の絞り込み項目及び値

例 :

```
Binding(
    semantic_path="$.invoice.invoiceLine.invoiceLineIdentifier",
    xpath="/Invoice/cac:InvoiceLine/cbc:ID",
    field="InvoiceLineIdentifier",
    dimension="dInvoiceLine"
)
```

build_binding_class_tree(rows) は、**type=C** の行から **BindingClass** ノードのツリーを構築します。

各 **BindingClass** は、次を保持します。

- **semantic_path**
- **xpath**
- **column** : クラス列
- **dimension** : **dInvoiceLine** などのディメンション列
- **repeats** : **multiplicity** に基づく繰返し状態
- **children**: list[**BindingClass**]

クラスツリーは、変換時に使用するセマンティックコンテキスト及びXMLコンテキストのツリーです。入力XMLから推定するのではなく、結合表から構築します。

direct_class_fields(class_path, bindings) は、ファクト結合を直接のセマンティック親ごとにグループ化します。結果は次の形式で使します。

```
direct_fields_by_class: dict[str, list[Binding]]
```

これにより、親クラスが子孫のファクトを早い段階で誤って抽出することを防ぎます。

ディメンション所有関係の解決

順方向変換と逆方向変換は、構文結合表から生成した同じ所有関係情報を使用します。

1. **multiplicity_repeats(multiplicity)** が、*、n、unbounded 又は2以上の数値上限を繰返しとして認識する。
2. **build_layout_from_rows(rows)** がすべての **type=C** クラス行を読み込む。請求書ルートは **dInvoice** となり、その他のクラスは多重度が繰返しの場合だけ **dXxx** となる。
3. 各 **type=A** 行を読み込む際、内部の **nearest_dimension(...)** が **semantic_path** を上位へたどり、ディメンションを所有する最も近いクラスを探す。
4. 結果を **BindingLayout.field_dimension** に保存し、**read_bindings(...)** が正規化した各 **Binding.dimension** へコピーする。

したがって、非繰返しの子クラスは、最も近い繰返し祖先の行に属します。例えば、繰返しの Invoice Line配下にある非繰返しのItem Informationは **dInvoiceLine** に属します。繰返しの子クラスは自身のディメンションを持ち、親行へ統合しません。

所有関係は、出力先XMLのQNameには依存しません。**type=C**、**multiplicity** 及び **semantic_path** によって決まり、実際のファクト列名及びクラス列名には **structured_csv_column** を使用します。

XPathコンテキストの再帰処理

順方向変換では、XMLの親コンテキストを再帰的に処理します。**write_hierarchical_csv(...)** 内の中心的な内部関数は、次のとおりです。

```
process_class(context, class_node, dimension_values, current_row)
```

引数：

- **context: ET.Element**：現在のクラスに対応するXML要素
- **class_node: BindingClass**：処理中のセマンティッククラス
- **dimension_values: dict[str, str]**：親コンテキストから引き継いだ現在のディメンション値
- **current_row: dict[str, str] | None**：現在の構造化CSV行

ルートでは、次のように呼び出します。

```
process_class(root_context, class_root, {"dInvoice": "1"}, None)
```

これは、最初の請求書に **dInvoice=1** を設定することを意味します。

A行の値設定

各クラスコンテキストで、**fill_direct_fields(row, context, class_node)** が、そのクラスに直接属する **A** 行だけ処理します。

各 **Binding** について、次を実行します。

1. **relative_xpath(binding.xpath, class_node.xpath)** が、絶対XPathを現在のクラスコンテキストからの相対パスへ変換する。結合先がそのコンテキスト外にある場合は絶対パスのままにし、文書ルートから評価する。
2. **get_value(context, relative_xpath, namespaces, root)** が、テキスト又は属性値を抽出する。
3. 値を **row[binding.field]** へ書き込む。

例：

クラス：

```
$.invoice.invoiceline  
/Invoice/cac:InvoiceLine
```

ファクト：

```
$.invoice.invoiceline.invoiceLineIdentifier  
/Invoice/cac:InvoiceLine/cbc:ID
```

相対XPath：

```
cbc:ID
```

変換プログラムは、現在の **cac:InvoiceLine** コンテキストから **cbc:ID** を抽出し、**InvoiceLineIdentifier** に書き込みます。

繰返しディメンションの判定

繰返しディメンションは、クラス行の **multiplicity** から導出した **BindingClass.repeats** によって決まります。

process_class(...) が子クラスを処理する際は、次のように動作します。

- 子が非繰返しの場合、同じ **current_row** を使って子进行处理する。
- 子が繰返しでディメンションを持つ場合、直ちに処理せず **repeated_children** に保持する。

現在の行に値を設定し、必要に応じて出力した後、繰返し子を出現単位で処理します。

繰返し子の各出現について、次を行います。

1. **class_contexts(parent_context, parent_class, child_class)** が、現在の親XMLコンテキスト配下の子XMLコンテキストを取得する。
2. 出現番号を1から数える。
3. 親ディメンションをコピーして、新しい **child_dimensions** 辞書を作る。

4. 子ディメンションの値を設定する。
5. 新しい行を作る。
6. 子XMLコンテキストへ再帰する。

例：

```
親ディメンション：
{
  "dInvoice": "1"
}

2番目のInvoice Line:
{
  "dInvoice": "1",
  "dInvoiceLine": "2"
}

そのInvoice Line配下の1番目のItem Attribute:
{
  "dInvoice": "1",
  "dInvoiceLine": "2",
  "dItemAttributes": "1"
}
```

このようにして、**dInvoice**、**dInvoiceLine** 及びそれより深いディメンション値を決定します。値は、現在のXML親コンテキスト内における1始まりの出現番号です。

行の生成及び疎な行

行は、次の辞書として表します。

```
dict[str, str]
```

new_row(fieldnames, d_invoice) は、すべての列を空欄で初期化し、**dInvoice** だけを設定した行を作ります。

row_has_values(row, dimension_columns) は、ディメンション値しかない空の行が追加されることを防ぎます。

最終出力は、次の形式です。

```
rows: list[dict[str, str]]
```

BindingLayout の **fieldnames** を使用し、**csv.DictWriter** で書き出します。

繰返し子がある場合、**process_class(...)** は **repeated_children** を処理する前に親行を完成させ、追加します。その後、1番目を含む各子出現について **new_row(...)** を呼び出します。このため、1番目の子も親行と同じ行には入りません。

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

dBbb が非繰返しの場合は、**process_class(...)** が同じ **current_row** を子へ渡すため、次のようになります。

```
1,,a1V1,a2V1,b1V1,b2V1,b3V1
```

Phase 1逆方向変換：構造化CSVからUBL XMLへ

スクリプト：

```
src/syntax_binding.py --reverse
```

主な関数：

```
write_xml_from_hierarchical_csv(...)
```

入力：

- 構造化CSV
- 順方向変換と同じUBL構文結合表

出力：

- UBL Invoice XML

逆方向データの解釈

逆方向モードでは、次を読み込みます。

```
rows: list[dict[str, str]]
bindings: list[Binding]
```

構造化CSVの各行は疎な行です。ディメンション列によって、その行がXML階層のどこに属するかを決定します。

例：

```
dInvoice=1
dInvoiceLine=2
InvoiceLineNetAmount=1000
```

これは、値が2番目の **cac:InvoiceLine** に属することを表します。

XMLを生成する前に、**validate_hierarchical_row_scopes(rows, fieldnames, field_dimension)** が、**build_layout_from_rows(...)** で作成した所有関係情報を適用します。

1. **is_dimension_column(...)** でディメンション列を識別する。
2. 各入力行について、値が入っている最も深いディメンションを行スコープとする。
3. 値が入っている各ファクトについて、**BindingLayout.field_dimension** から所有ディメンションを取得する。
4. 所有ディメンションと行スコープが異なる場合は **ValueError** を発生させる。

これにより、**1,1,a1V1,a2V1,b1V1,b2V1,b3V1** のように親ファクトと繰返し子ファクトを混在させた行を拒否します。親ファクトは親行に残し、繰返し子のファクトは独立した子行から開始する必要があります。

XMLの構築

逆方向出力では、次のルート要素を作成します。

```
root: ET.Element
```

その後、結合表のXPathを使って値を書き込みます。

主な補助関数：

- **ensure_path(root, xpath, namespaces, force_new_leaf=False)**：XPathをたどり、不足するXML要素を作成する。
- **find_or_create_child(parent, step, namespaces, force_new=False)**：タグ及び述語に一致する子要素を探し、存在しなければ作成する。
- **create_context(...)**：繰返し行に対応するXML親コンテキストを作成又は再利用する。
- **set_xml_value_with_currency(...)**：値を書き込み、出力先が金額の場合は **currencyID** を設定する。
- **set_relative_xml_value(...)**：現在の繰返し構文コンテキスト内に含まれるパスへ値を書き込む。
- 繰返しコンテキスト外に残る絶対XPathは、文書ルートを指定して **set_xml_value_with_currency(...)** へ渡す。例えばBT-90は、セマンティックパス上では

Payment Instructions配下にある一方、UBL XPathは **AccountingSupplierParty** 配下にあるため、この方式で処理する。

- **ensure_tax_scheme_defaults(...)** : UBLで必要なTax Schemeの既定要素を追加する。
- **load_ubl_child_order(...)** : **--ubl-schema-root** 又は **--ubl-schema-url** からUBL XSDを読み込み、**xs:sequence** から子要素順を構築する。
- **sort_children_for_ubl_schema(...)** : 生成したUBLスキーマ順に子要素を並べ替える。スキーマの参照元を指定しない場合は、現在のPoC構造に対応する組み込み順序を使用する。

逆方向処理は、パスを構築する方式です。同じ結合表を使用しますが、現時点では順方向処理と同じ再帰的な **BindingClass** ツリーは使用しません。繰返しコンテキストのXPathに含まれる結合XPathだけを相対パスへ変換し、コンテキスト外の絶対パスは文書ルートから処理します。スキーマ順を考慮しており、スキーマルート又はスキーマURLを指定すれば、UBL 2.1以降のUBL XSD順を使用できます。